TRAVAUX PRATIQUES

Java RMI

Fabrique d'objets · Callbacks · Passage par copie et par référence

Prérequis	Java SE, interfaces, sérialisation, notions de réseau
Outils	JDK 11+, Maven ou Gradle, IDE (IntelliJ / Eclipse), terminal
Rendu	Archive ZIP : sources + rapport PDF (schémas + observations)

🎯 **Objectifs pédagogiques :** À l'issue de ce TP, vous serez capable d'implémenter le patron Fabrique via RMI, de gérer des callbacks distribués (patron Observer), et de maîtriser la distinction entre passage d'objets par valeur (Serializable) et par référence (Remote). Vous saurez également organiser un projet RMI multi-modules et tester la tolérance aux pannes basique.

0. Contexte et Architecture RMI

0.1 Rappel du modèle RMI

Java RMI (*Remote Method Invocation*) permet à une JVM cliente d'invoquer des méthodes d'objets résidant dans une JVM distante. L'illusion de transparence est assurée par deux composants générés automatiquement :

- **Stub** Le
  (côté client)
  - est un proxy local qui intercepte les appels, sérialise les arguments et les envoie via JRMP.

- **Skeleton** Le
  (côté serveur, implicite depuis Java 5)
  - désérialise les arguments, invoque la méthode réelle et renvoie le résultat.

Le **rmiregistry** (port 1099 par défaut) joue le rôle d'annuaire : le serveur y enregistre ses objets distants (`bind` / `rebind`), et le client les localise (`lookup`).

0.2 Structure de projet recommandée

Organisez votre projet Maven en trois modules ou packages distincts :

```
tp-rmi/
├── src/main/java/
│   ├── interfaces/     ← Interfaces Remote partagées (+ Serializable DTOs)
│   ├── server/         ← Implémentations + main serveur
│   └── client/         ← Clients + implémentations callback
└── pom.xml
```

⚠️ **Attention :** Toute interface d'objet distant doit étendre java.rmi.Remote. Chaque méthode doit déclarer throws RemoteException. Les implémentations concrètes d'objets distants doivent étendre UnicastRemoteObject.

Partie 1 — Fabrique d'objets distants

1.1 Présentation

Le patron **Factory** appliqué à RMI permet à un client d'obtenir dynamiquement des objets distants sans connaître leur classe d'implémentation. Le client invoque une méthode de création sur la fabrique (objet distant), qui instancie et retourne un stub vers le nouvel objet.

1.2 Interfaces à implémenter

Créez les interfaces suivantes dans le package interfaces :

```
// Interface de la forme distante
public interface Shape extends Remote {
    double area()        throws RemoteException;
    double perimeter()   throws RemoteException;
    String describe()    throws RemoteException;
}

// Interface de la fabrique distante
public interface ShapeFactory extends Remote {
    Shape createShape(String type, double... params) throws RemoteException;
    String[] availableTypes()                        throws RemoteException;
}
```

1.3 Travail demandé

Tâche 1.1 — Implémentations de Shape

Implémentez les classes suivantes (chacune étend UnicastRemoteObject) :

- CircleImpl(double radius) : aire = $\pi \cdot r^2$, périmètre = $2\pi \cdot r$
- RectangleImpl(double w, double h) : aire = $w \times h$, périmètre = $2(w+h)$
- TriangleImpl(double a, double b, double c) : formule de Héron, périmètre = $a+b+c$

Tâche 1.2 — ShapeFactoryImpl

Implémentez ShapeFactoryImpl qui crée l'objet approprié selon le paramètre type ("circle", "rectangle", "triangle"). Retournez une RemoteException descriptive pour les types inconnus.

Tâche 1.3 — Serveur et client

Écrivez un Main serveur qui enregistre la fabrique dans le registry, et un client qui :

1. récupère la fabrique via registry.lookup("ShapeFactory")
2. crée un cercle, un rectangle et un triangle via la fabrique
3. affiche pour chaque forme son type, son aire et son périmètre

Tâche 1.4 — Extension (bonus)

Ajoutez une classe PentagonImpl. Modifiez le client pour appeler availableTypes() et afficher les formes disponibles avant de les créer.

Partie 2 — Callbacks RMI

2.1 Présentation

Les callbacks RMI implémentent le patron **Observer** distribué : un client s'enregistre auprès du serveur, et ce dernier rappelle activement les clients lors d'événements. Le point clé est que **le client doit lui-même exporter un objet Remote** pour être rappelable.

Flux : client appelle `register(callbackObj)` → serveur stocke le stub → événement → serveur itère et appelle `onNotify(event)` sur chaque stub.

2.2 Interfaces à implémenter

```
// Interface callback — exportée par le CLIENT
public interface GameCallback extends Remote {
    void onPlayerJoined(String playerName)  throws RemoteException;
    void onGameEvent(String eventMessage)   throws RemoteException;
    void onGameOver(String winner)           throws RemoteException;
}

// Interface du serveur de jeu
public interface GameServer extends Remote {
    String joinGame(String playerName, GameCallback cb) throws RemoteException;
    void   sendAction(String playerId, String action)   throws RemoteException;
    void   leaveGame(String playerId)                   throws RemoteException;
    int    getPlayerCount()                              throws RemoteException;
}
```

2.3 Travail demandé

Tâche 2.1 — GameCallbackImpl

Implémentez GameCallbackImpl côté client, qui étend UnicastRemoteObject et imprime chaque notification reçue avec un préfixe horodaté (ex : [14:32:05] NOTIF : ...).

Tâche 2.2 — GameServerImpl

Implémentez GameServerImpl avec :

- Une Map<String, GameCallback> pour stocker les joueurs enregistrés
- La méthode joinGame qui génère un playerId unique, stocke le callback, notifie TOUS les autres joueurs de la nouvelle connexion, et retourne le playerId
- La méthode sendAction qui diffuse l'action à tous les joueurs
- La gestion des RemoteException dans la diffusion : si un client est déconnecté, le retirer silencieusement de la liste

Tâche 2.3 — Simulation multi-clients

Écrivez un client qui simule 3 joueurs : chacun dans un Thread séparé, se connecte, envoie 2 actions aléatoires, attend 2 secondes, puis se déconnecte. Vérifiez que chaque joueur reçoit les notifications des autres.

Tâche 2.4 — Résilience (bonus)

Testez la déconnexion brutale d'un client (arrêt forcé de sa JVM) et vérifiez que le serveur ne plante pas lors de la prochaine diffusion.

Partie 3 — Passage d'objets en paramètres (5 points)

3.1 Passage par copie (Serializable)

Un objet est passé **par copie** quand il implémente `java.io.Serializable` sans implémenter `Remote`. À l'appel, la JVM sérialise l'objet, le transmet sur le réseau, et la JVM distante le désérialise en une copie indépendante. Toute modification côté serveur sur cette copie est invisible côté client, et inversement.

3.2 Passage par référence (Remote)

Un objet est passé **par référence distante** quand il implémente `Remote` et étend `UnicastRemoteObject`. À l'appel, la JVM transmet un **stub** (proxy) pointant vers l'objet original. Toute modification via ce stub se répercute sur l'objet réel.

3.3 Travail demandé

Tâche 3.1 — Démonstration par copie

Créez la classe Vector2D implements Serializable avec des champs double x, y et une méthode translate(dx, dy). Créez une interface VectorService extends Remote avec :

```
double magnitude(Vector2D v)              throws RemoteException;
Vector2D normalize(Vector2D v)             throws RemoteException;
Vector2D add(Vector2D a, Vector2D b)       throws RemoteException;
```

Testez et observez : dans le client, modifiez v.x après l'appel. La valeur sur le serveur ne change pas. Documentez cette observation dans votre rapport.

Tâche 3.2 — Démonstration par référence

Créez l'interface SharedCounter extends Remote avec increment(), decrement(), getValue() et reset(). Implémentez SharedCounterImpl extends UnicastRemoteObject. Créez un service orchestrateur :

```
public interface CounterService extends Remote {
    SharedCounter createCounter(String name) throws RemoteException;
    // SharedCounter est Remote → passé par référence (stub)
    void atomicIncrement(SharedCounter c, int n) throws RemoteException;
    int  sum(SharedCounter c1, SharedCounter c2) throws RemoteException;
}
```

Testez : créez deux compteurs, incrémentez-les via atomicIncrement, vérifiez que sum() retourne la somme correcte. Confirmez que les modifications via le stub sont visibles en temps réel.

Partie 4 — Projet intégrateur : Gestionnaire de tâches distribué

4.1 Description

Implémentez un **gestionnaire de tâches distribué** combinant les trois mécanismes étudiés. Les clients peuvent créer des tâches, les assigner et suivre leur progression en temps réel.

4.2 Contraintes obligatoires

- Utilisez une fabrique pour créer les tâches (Task factory pattern)
- Les TaskData (titre, description, priorité) sont des Value Objects Serializable
- Les TaskHandle (contrôle de progression) sont des objets Remote passés par référence
- Les clients s'abonnent aux mises à jour via callbacks
- Au moins 3 états de tâche : PENDING → IN_PROGRESS → DONE

4.3 Interfaces minimales

```
// Value Object (par copie)
public class TaskData implements Serializable {
    String title, description;
    int priority; // 1=LOW, 2=MEDIUM, 3=HIGH
}

// Objet de contrôle (par référence)
public interface TaskHandle extends Remote {
    void   updateProgress(int percent) throws RemoteException;
    void   complete(String result)      throws RemoteException;
    String getStatus()                  throws RemoteException;
}

public interface TaskManager extends Remote {
    TaskHandle createTask(TaskData data)          throws RemoteException;
    void       subscribe(TaskCallback cb)          throws RemoteException;
    List<String> listPendingTasks()                throws RemoteException;
}

public interface TaskCallback extends Remote {
    void onTaskCreated(TaskData data)              throws RemoteException;
    void onTaskCompleted(String taskId, String res)throws RemoteException;
}
```

Annexe — Commandes et aide-mémoire

Compilation et lancement

```
# Compiler le projet
javac -d target/classes src/main/java/**/*.java

# Démarrer le registry (dans un terminal séparé)
rmiregistry 1099 &

# Lancer le serveur
java -cp target/classes server.MainServer

# Lancer un client
java -cp target/classes client.MainClient localhost

# Avec Maven
mvn compile exec:java -Dexec.mainClass="server.MainServer"
```

Erreurs fréquentes

Erreur	Cause / Solution
`java.rmi.ConnectException`	Le registry ou le serveur n'est pas démarré. Vérifiez rmiregistry 1099.
`java.rmi.NotBoundException`	Le nom utilisé dans lookup() ne correspond pas au nom enregistré avec bind/rebind.
`java.io.NotSerializableException`	Un objet passé en paramètre ou retourné n'implémente pas Serializable.
`java.rmi.UnmarshalException`	Classes manquantes côté client. Vérifiez que les interfaces sont dans le classpath.
`ConcurrentModificationException (callbacks)`	Utilisez CopyOnWriteArrayList pour itérer pendant une suppression.

Squelette pom.xml

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>tn.utm.rmi</groupId>
  <artifactId>tp-rmi</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>
</project>
```